

HTTP notes for webserv

The subset of HTTP/1.1 we actually need for webserv, with examples of valid and invalid input. Not a full spec. For edge cases see [RFC 9112](#).

1. Request structure

Every request has the same shape: request line, headers, an empty line, then an optional body.

```
GET /index.html HTTP/1.1\r\n    <- request line
Host: localhost:8080\r\n      <- header
User-Agent: curl/8.5\r\n      <- header
\r\n                          <- empty line, end of headers
[body, if any]
```

Sequence	Meaning	Where
\r\n	end of one line	after the request line and after every header
\r\n\r\n	last line ends + empty line = end of headers	every request
0\r\n\r\n	zero-size chunk = end of body	chunked requests only

The end of the body is never marked by an empty line, because the body can contain any bytes. Its length comes from `Content-Length` or from chunked encoding (section 4).

2. Request line

Exactly three parts separated by single spaces: `METHOD SP target SP version\r\n`.

- Method: we support `GET`, `POST`, `DELETE`. Case-sensitive, so `get` is an unknown method.
- Target: a path starting with `/`, may include a query: `/cgi-bin/x.py?name=anna`.
- Version: `HTTP/1.1` (accepting `HTTP/1.0` too is common).

Example	Result
<code>GET / HTTP/1.1</code>	valid
<code>POST /upload HTTP/1.1</code>	valid
<code>GET /search?q=cat HTTP/1.1</code>	valid, query is part of the target
<code>GET / HTTP/1.1</code> (two spaces)	400
<code>GET /</code> (no version)	400
<code>GET index.html HTTP/1.1</code> (no leading <code>/</code>)	400
<code>PUT / HTTP/1.1</code>	501, valid HTTP method we don't implement
<code>DELETE /x HTTP/1.1</code> where the location allows only GET	405, decided by response using config, not by the parser

Example	Result
GET / HTTP/2.0	505
GET / HTTP/1.1	400

3. Headers

Format: `Name: value\r\n`.

- Names are case-insensitive: `Content-Length`, `content-length` and `CONTENT-LENGTH` are the same header. The parser stores names in lowercase.
- Whitespace around the value is trimmed: `Host: localhost` gives `localhost`.
- No whitespace allowed between the name and `:`.
- `Host` is required in HTTP/1.1, exactly once.
- Unknown headers (`X-Whatever: 1`) are fine, just store them.

Example	Result
<code>Host: localhost</code>	valid
<code>host:localhost</code>	valid
<code>X-Custom: abc</code>	valid
<code>Host : localhost</code> (space before <code>:</code>)	400
<code>Host localhost</code> (no <code>:</code>)	400
HTTP/1.1 request without <code>Host</code>	400
two <code>Host</code> headers	400

4. Body: where it ends

After `\r\n\r\n` the parser looks at two headers to decide how much more to read.

Content-Length	Transfer-Encoding	What to do
absent	absent	no body, request is complete right after <code>\r\n\r\n</code>
<code>N</code>	absent	read exactly N bytes
absent	<code>chunked</code>	read chunks until <code>0\r\n\r\n</code> (section 5)
<code>N</code>	<code>chunked</code>	400, conflicting framing (this is what request smuggling exploits)
any	<code>gzip</code> or anything else	501

`Content-Length` rules:

- It counts bytes, not characters. `привіт` in UTF-8 is 12 bytes.
- Digits only. `abc`, `-5`, `5.0` give 400.

- Two `Content-Length` headers with different values give 400.
- Larger than `client_max_body_size` from the config gives 413. With `Content-Length` you know this before reading the body.
- A GET with a body is technically allowed and pointless, but the body still has to be consumed. Otherwise those bytes get parsed as the start of the next request.

More bytes than N arrived: the rest is the next request (keep-alive). Leave it in the buffer, don't drop it.

Fewer bytes than N arrived: not an error, just `NeedMoreData`. The rest comes with the next `recv`. If it never comes, the timeout closes the connection.

5. Chunked format

The body is sent in pieces. Each chunk is: size in hex, `\r\n`, data, `\r\n`. A chunk of size `0` ends the body.

```
POST /cgi-bin/x.py HTTP/1.1\r\n
Host: localhost\r\n
Transfer-Encoding: chunked\r\n
\r\n          <- end of headers, same as any request
5\r\n        <- size 5
hello\r\n    <- 5 bytes of data + \r\n
A\r\n        <- size 10 (hex A)
, world!!!\r\n
0\r\n        <- size 0, no more data
\r\n         <- empty line, request complete
```

Decoded body: `hello, world!!!` (15 bytes). The sizes and the `\r\n` between chunks are framing and don't go into the body.

Fragment	Result
<code>5\r\nhello\r\n0\r\n\r\n</code>	valid
<code>a\r\n...</code> or <code>A\r\n...</code>	valid, hex in either case
<code>5;foo=bar\r\nhello\r\n</code>	valid, ignore everything after <code>;</code>
<code>0\r\nX-Trailer: 1\r\n\r\n</code>	valid, trailer lines are read and ignored
<code>5\r\nhello0\r\n\r\n</code> (no <code>\r\n</code> after data)	400
<code>zz\r\n...</code> (not hex)	400
<code>5\r\nhel</code> and then nothing	<code>NeedMoreData</code> , then timeout
total size over <code>client_max_body_size</code>	413, check after every chunk since the total isn't known upfront

6. Status codes and who detects them

The parser catches format errors (the request itself is broken). Response catches meaning errors (the request is fine, but config or the filesystem says no). After a parser error, send the response and close the connection: we can't tell where the next request starts.

Code	When	Detected by
400 Bad Request	broken request line or header, missing <code>Host</code> , CL + chunked, bad chunk	parser
413 Content Too Large	body over <code>client_max_body_size</code>	parser (limit from config)
501 Not Implemented	unknown method, <code>Transfer-Encoding</code> other than chunked	parser
505 HTTP Version Not Supported	version other than 1.0 / 1.1	parser
404 Not Found	file doesn't exist	response
403 Forbidden	no permission, or directory with no index and autoindex off	response
405 Method Not Allowed	method not in the location's <code>allowed_methods</code>	response
408 Request Timeout	client didn't finish sending the request in time	event loop / connection
500 Internal Server Error	CGI crashed or returned garbage	CGI
504 Gateway Timeout	CGI ran too long	CGI / event loop

7. Response

Same shape as a request, with a status line instead of a request line.

```
HTTP/1.1 200 OK\r\n
Content-Type: text/html\r\n
Content-Length: 12\r\n
\r\n
<h1>Hi!</h1>
```

- Always set `Content-Length` , 0 for an empty body. That's how the client knows the response is over.
- Errors are responses too: `HTTP/1.1 404 Not Found` with an error page from the config or a default HTML page.
- No need for chunked responses.

Keep-alive (optional)

Keep-alive means the connection stays open after the response so the next request can reuse it. HTTP/1.1 assumes it by default, but closing after every response is also valid HTTP.

Without keep-alive: close the connection after every response and add `Connection: close` to it. After writing is done, go straight to closing, no return to reading.

With keep-alive: after writing is done, go back to reading on the same connection. Then:

- `Content-Length` in the response is mandatory. Without it the client can't see where the response ends and will wait until the connection closes.
- Leftover bytes in the parser buffer belong to the next request.
- An idle timeout is needed so dead connections don't stay open forever.
- Decide per request:

Version	Connection header in request	After the response
HTTP/1.1	absent	keep open
HTTP/1.1	<code>close</code>	close
HTTP/1.0	absent	close
HTTP/1.0	<code>keep-alive</code>	keep open (optional to support)
any		close after a parser error

8. Testing

`nc` lets you send broken requests that `curl` would never produce. `curl` is easier for valid ones.

```
# valid GET
printf 'GET / HTTP/1.1\r\nHost: localhost\r\n\r\n' | nc localhost 8080

# missing Host -> 400
printf 'GET / HTTP/1.1\r\n\r\n' | nc localhost 8080

# POST with Content-Length
curl -v -H "Expect:" --data 'hello' http://localhost:8080/cgi-bin/x.py

# POST chunked (curl encodes the body itself)
curl -v -H "Expect:" -H "Transfer-Encoding: chunked" --data 'hello' http://localhost:8080/cgi-bin/x.py

# chunked by hand
printf 'POST /cgi-bin/x.py HTTP/1.1\r\nHost: localhost\r\nTransfer-Encoding: chunked\r\n\r\n5\r\nhello\r\n0\r\n\r\n' | nc localhost 8080

# Content-Length + chunked -> 400
printf 'POST / HTTP/1.1\r\nHost: localhost\r\nContent-Length: 5\r\nTransfer-Encoding: chunked\r\n\r\n5\r\nhello\r\n0\r\n\r\n' | nc localhost 8080

# request split across two sends (NeedMoreData path)
(printf 'GET / HTTP/1.1\r\nHo'; sleep 1; printf 'st: localhost\r\n\r\n') | nc localhost 8080

# two requests on one connection (keep-alive)
printf 'GET / HTTP/1.1\r\nHost: localhost\r\n\r\nGET / HTTP/1.1\r\nHost: localhost\r\n\r\n' | nc localhost 8080

# same with curl: two URLs in one call reuse the connection ("Re-using existing connection" in -v output)
curl -v http://localhost:8080/ http://localhost:8080/
```

Why `-H "Expect:"` : for larger POST bodies curl sends `Expect: 100-continue` and waits for the server's go-ahead before sending the body. If the server doesn't answer it, curl waits about a second and sends the body anyway. The empty header turns that off.